NILS HARTMANN
https://nilshartmann.net



GraphQL

Verheißung oder Verhängnis?

Ein praktischer Deep Dive

SOFTWERKSKAMMER NÜRNBERG | 21. APRIL 2026 | @NILSHARTMANN

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Entwickler, Architekt, Trainer aus Hamburg

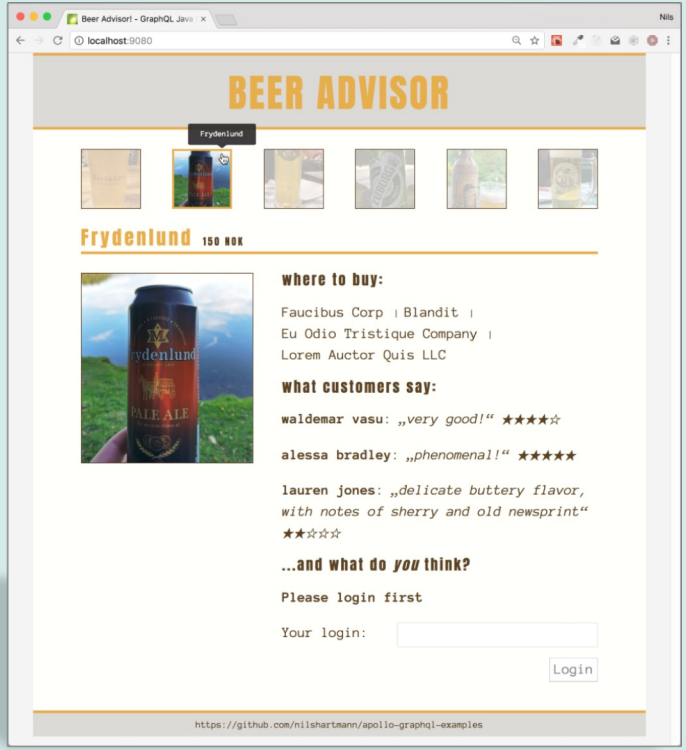
Java, Spring, GraphQL, TypeScript, React



https://graphql.schule/video-kurs



https://reactbuch.de



Beispiel Anwendung

Source: https://github.com/nilshartmann/spring-graphql-talk

Eine API für den Beeradvisor...

Ansatz 1: Backend bestimmt Aussehen der Endpunkte / Daten

/api/beer



/api/shop



/api/rating



Ansatz 1: Backend bestimmt Aussehen der Endpunkte / Daten

REST / HTTP APIs

/api/beer

Beer
id name price ratings shops

/api/shop

Shop
id name street city phone

/api/rating

Rating
id author stars comment

Ansatz 2: Client diktiert die API nach seinen Anforderungen

/api/home

```
id
name
avgStars
```

/api/beer-view

```
id
name
price
shopName
ratings
```

/api/shopdetails

```
id
shopName
shopStreet
shopCity
beerNames
```

Ansatz 2: Client diktiert die API nach seinen Anforderungen

Backend for Frontend (BFF)

/api/home

```
id
name
avgStars
```

/api/beer-view

```
id
name
price
shopName
ratings
```

/api/shopdetails

```
id
shopName
shopStreet
shopCity
beerNames
```

Ansatz 3: GraphQL...

Ansatz 3: GraphQL...

- Aus Ansatz 1: Server bestimmt, wie Datenmodell aussieht



Ansatz 3: GraphQL...

- Aus Ansatz 1: Server bestimmt, wie Datenmodell aussieht
- ...aber Client kann pro Ansicht wählen, welche Daten er daraus benötigt

```
{ beer { id price { shops { name } } } }
```

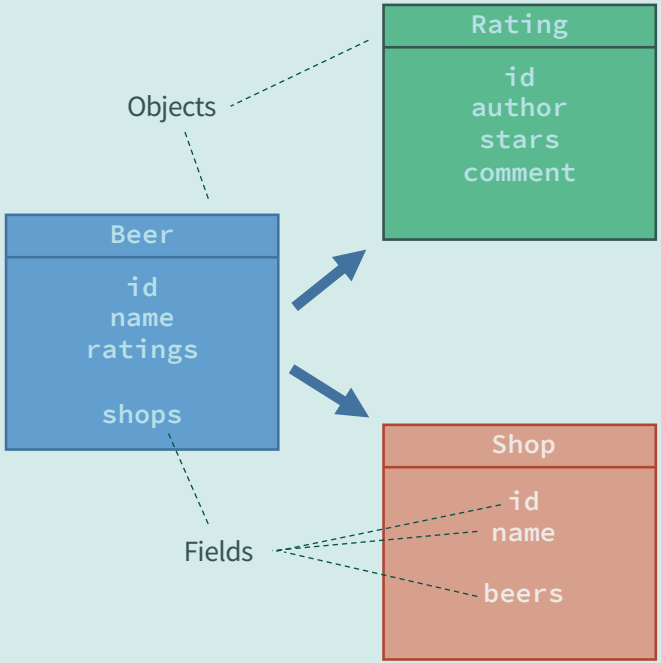


*"GraphQL is a **query language for APIs** and a **runtime for fulfilling those queries** with your existing data"*

- https://graphql.org

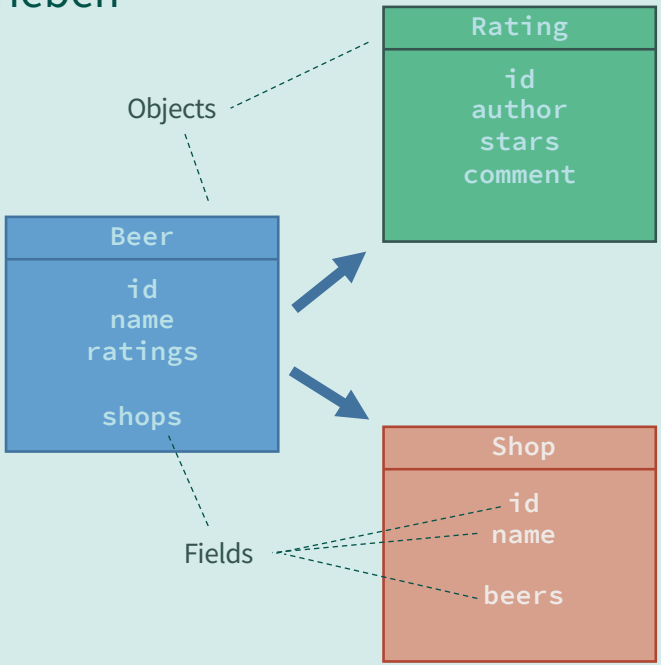
GraphQL

Über eine GraphQL API werden *Objekte* mit *Feldern* bereitgestellt



Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

- Die Objekte und Felder sind in einem **Schema** beschrieben



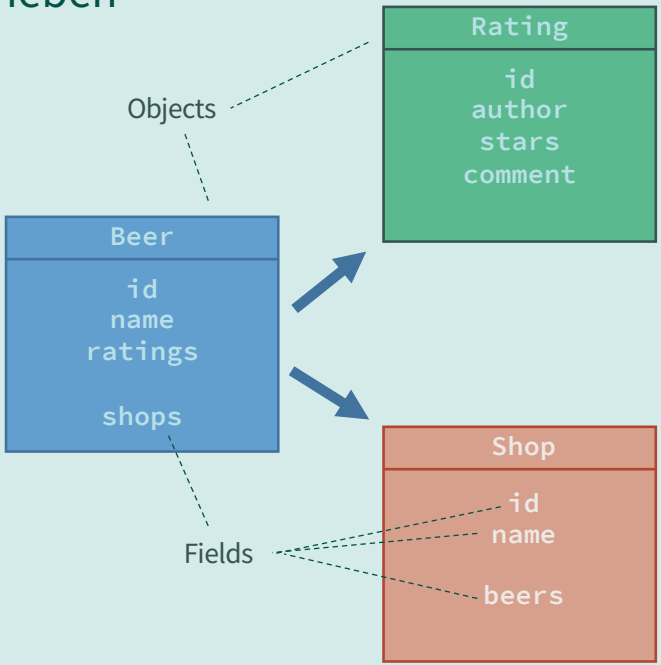
Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

- Die Objekte und Felder sind in einem **Schema** beschrieben
- Dafür gibt es ein eigenes **Typsystem**

```
type Beer {
  id: ID!
  name: String!
  ratings: [Rating!]!
  shops: [Shop!]
}
```

```
type Rating {
  id: ID!
  author: String!
  stars: Int!
  comment: String!
}
```

```
type Shop {
  id: ID!
  name: String!
  beers: [Beer!]!
}
```



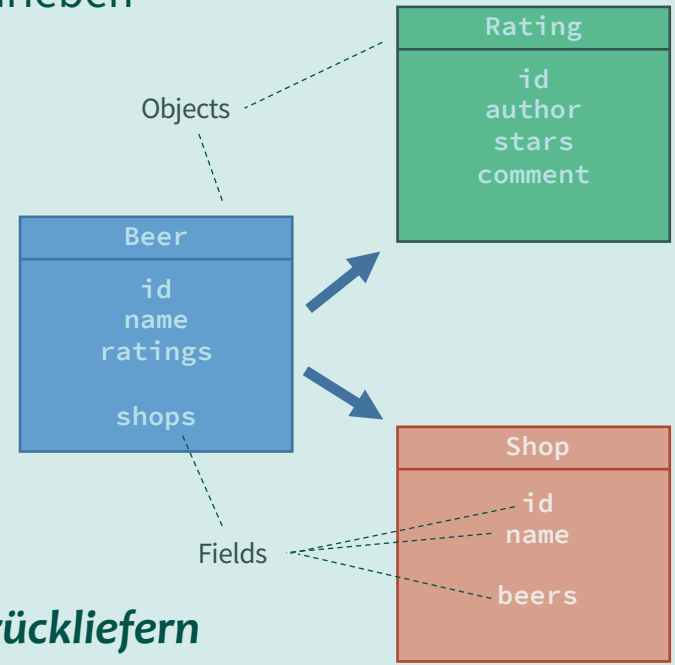
Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

- Die Objekte und Felder sind in einem **Schema** beschrieben
- Dafür gibt es ein eigenes **Typsystem**

```
type Beer {
  id: ID!
  name: String!
  ratings: [Rating!]!
  shops: [Shop!]
}
```

```
type Rating {
  id: ID!
  author: String!
  stars: Int!
  comment: String!
}
```

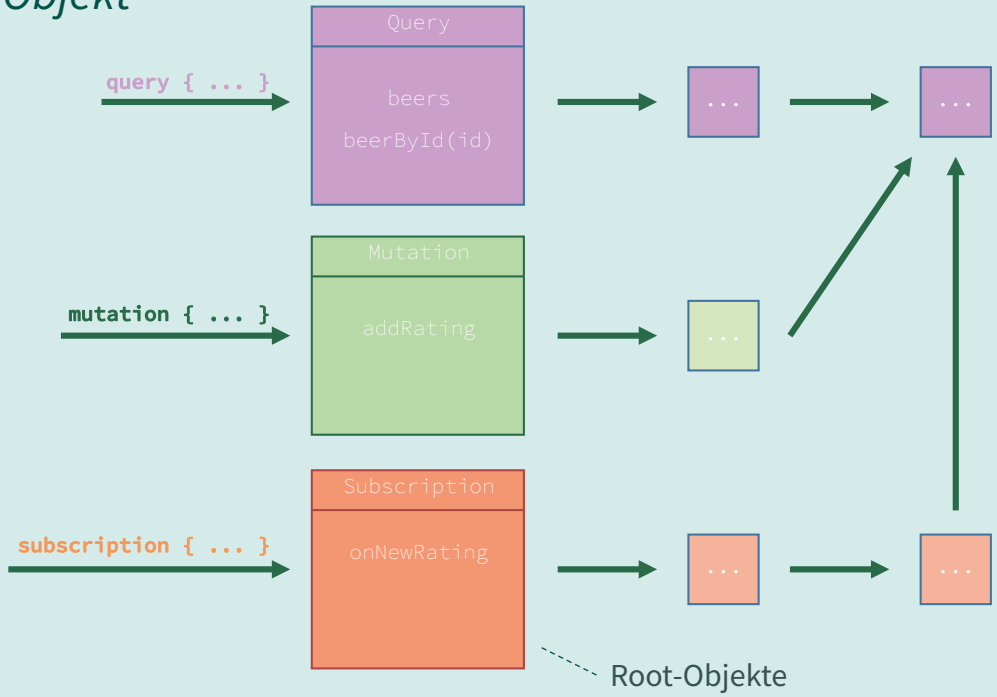
```
type Shop {
  id: ID!
  name: String!
  beers: [Beer!]!
}
```



👉 *Felder sind konzeptionell Funktionen, die Werte zurückliefern*

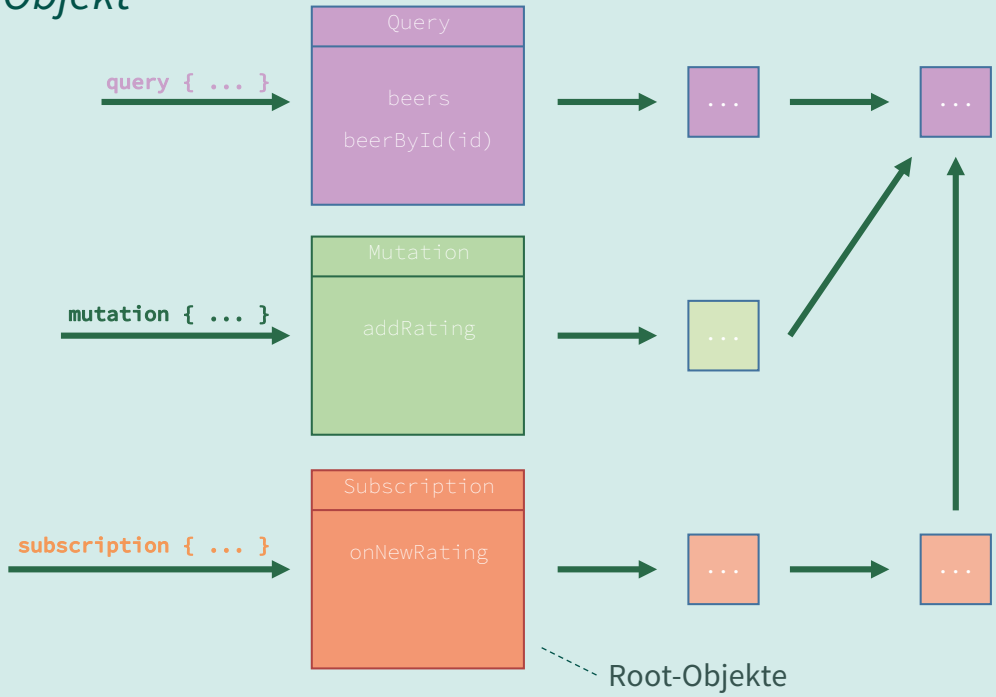
Über eine GraphQL API werden *Objekte* mit *Feldern* bereitgestellt

- Der Graph beginnt bei einem *Root-Objekt*



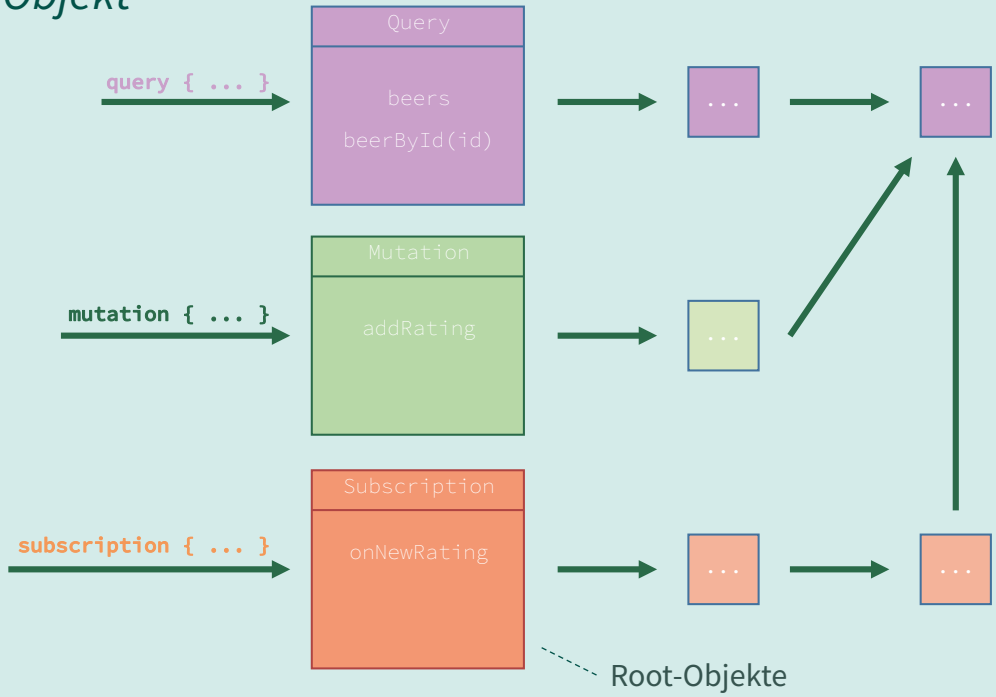
Über eine GraphQL API werden *Objekte* mit *Feldern* bereitgestellt

- Der Graph beginnt bei einem *Root-Objekt*
- Es gibt drei Root-Objekte*: **query**, **mutation** und **subscription***



Über eine GraphQL API werden *Objekte* mit *Feldern* bereitgestellt

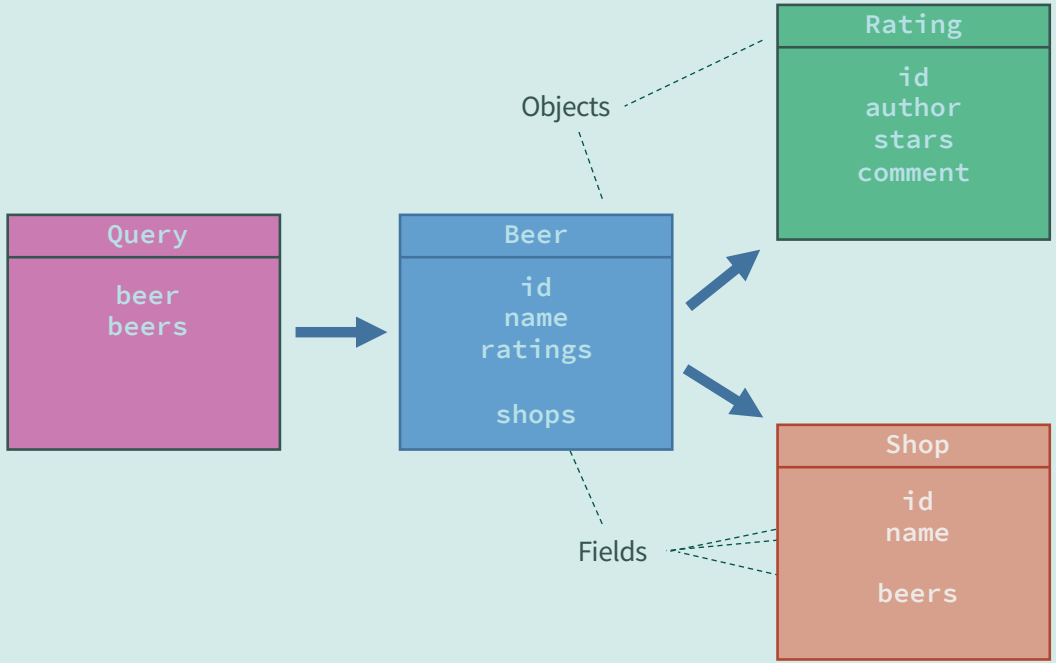
- Der Graph beginnt bei einem *Root-Objekt*
- Es gibt drei Root-Objekte*: **query**, **mutation** und **subscription***
- Jedes Root-Objekt ist ein eigener *Einstiegspunkt* in die Api



Die Query Sprache

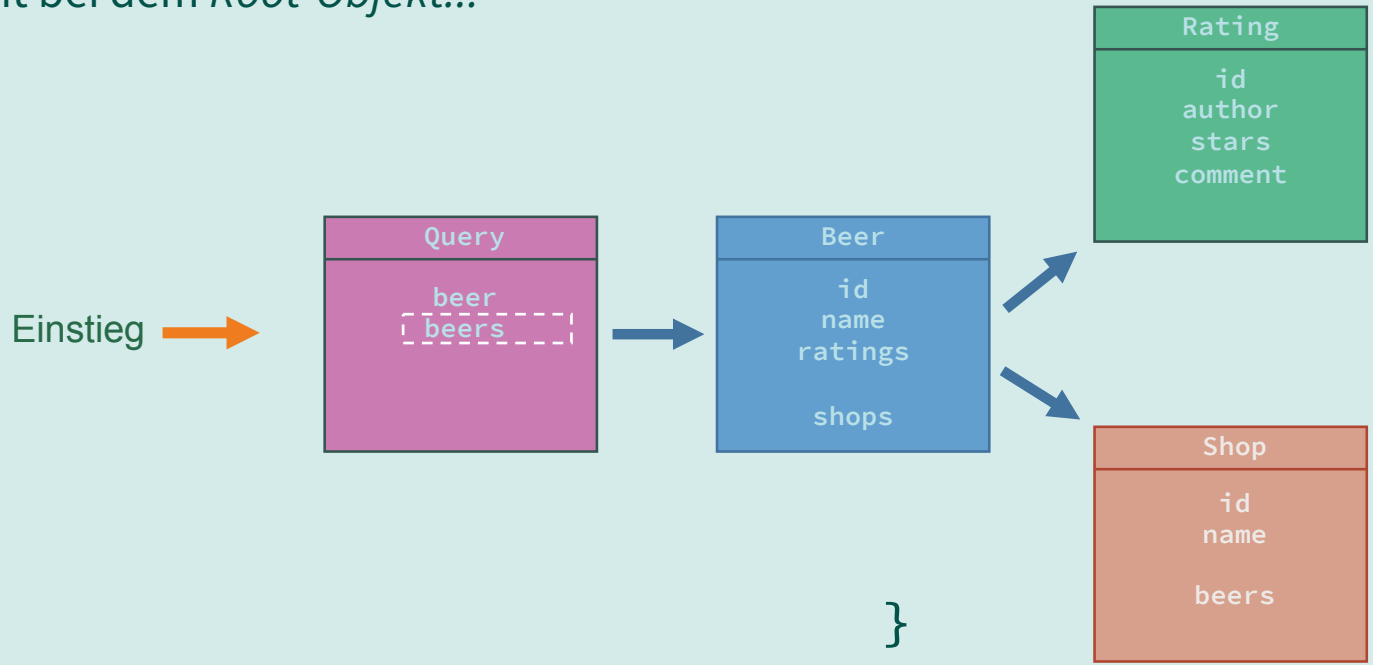
QUERY LANGUAGE

Mit der *Query-Sprache* werden aus dem Graphen *Felder* ausgewählt



Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

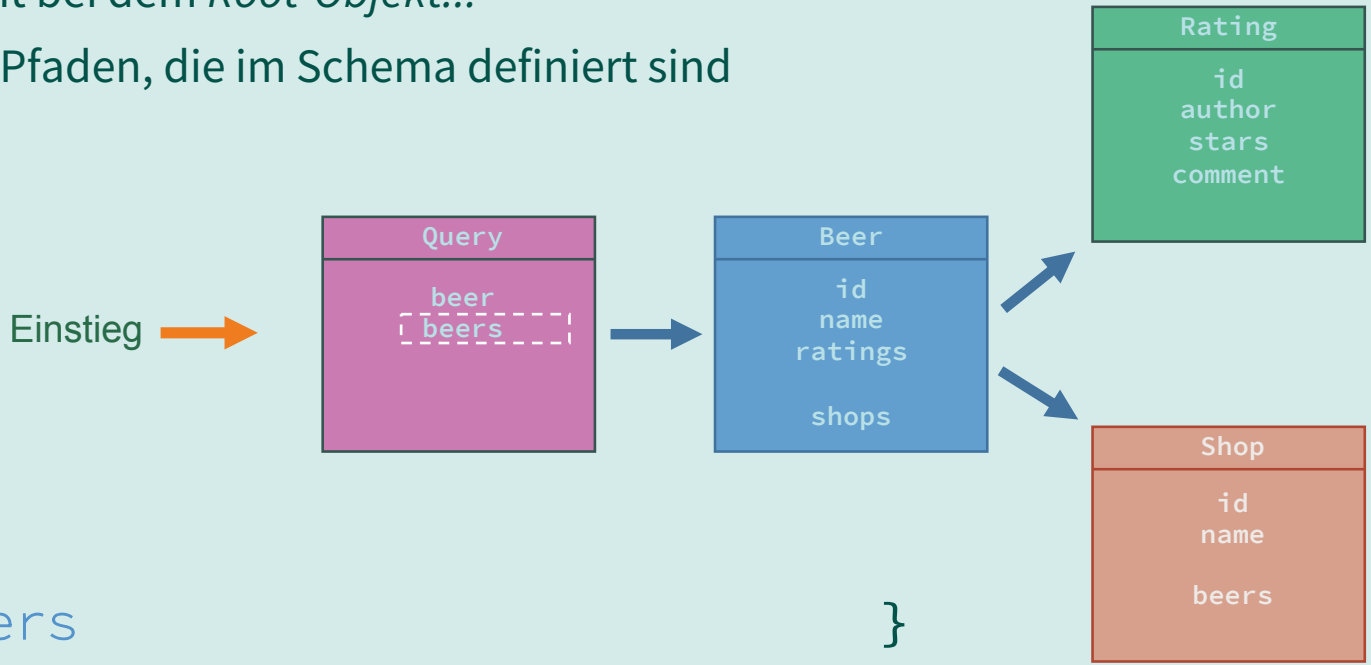
- Der Query beginnt bei dem *Root-Objekt*...




```
query {                                                    }
```

Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

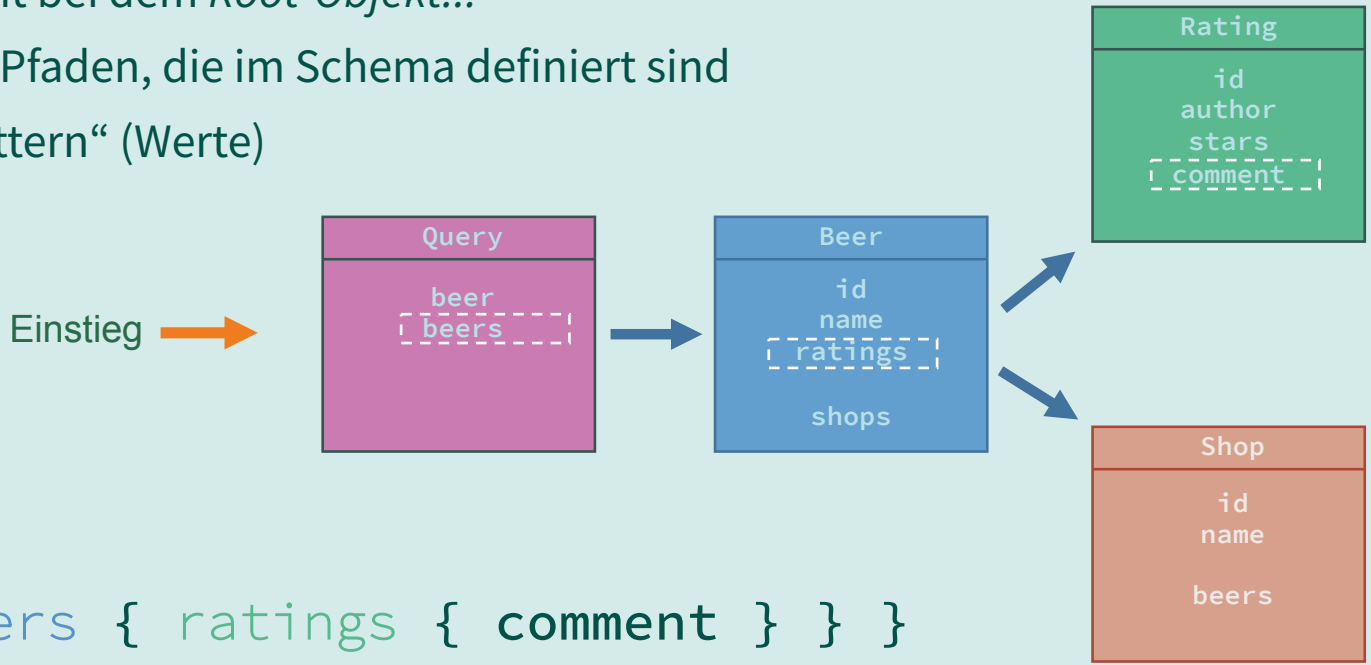
- Der Query beginnt bei dem *Root-Objekt*...
- ...folgt dann den Pfaden, die im Schema definiert sind




```
query { beers                                    }
```

Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

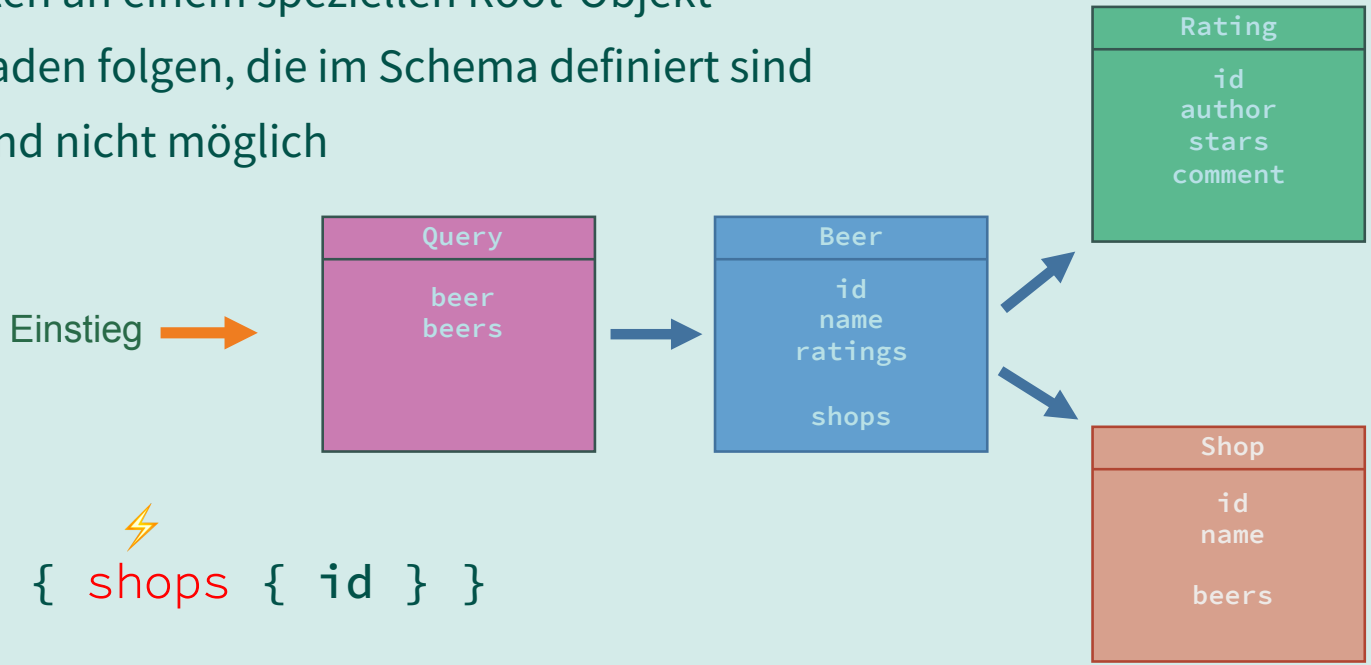
- Der Query beginnt bei dem *Root-Objekt*...
- ...folgt dann den Pfaden, die im Schema definiert sind
- ...bis zu den “Blättern“ (Werte)



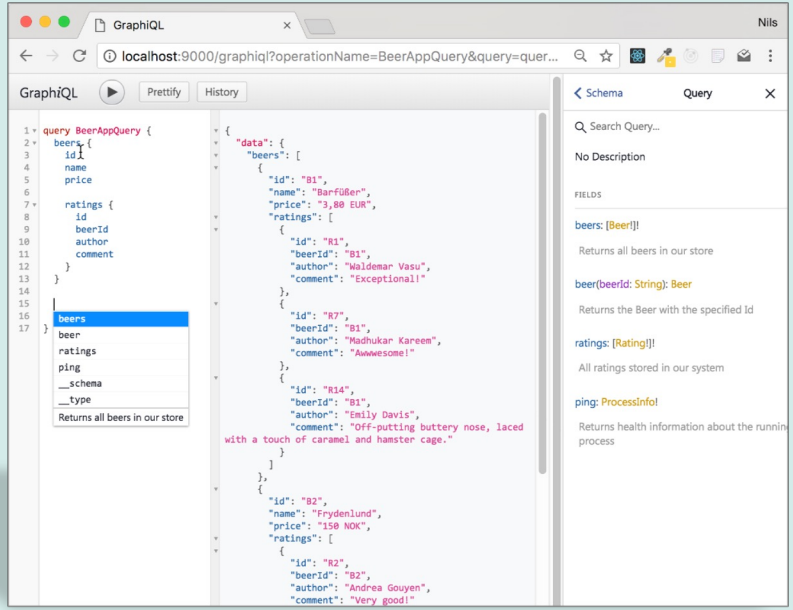

```
query { beers { ratings { comment } } }
```

Mit der Query-Sprache werden *Felder* von *Objekten* abgefragt

- Alle Queries starten an einem speziellen Root-Objekt
- Man kann nur Pfaden folgen, die im Schema definiert sind
- Andere "joins" sind nicht möglich




```
query { shops { id } }
```



- Ein Query
- Mutation
- Subscription
- Dokumentation
- Netzwerkverkehr (evtl.)

Demo: Query-Sprache

https://github.com/graphql/graphiql

Das Schema

- Es gibt nur eine Version

```
type Beer {
  name: String!
  price: Int!
  type: BeerType!
  created: DateTime
}
```

Das Schema

- Es gibt nur eine Version
- Schema kann jederzeit erweitert werden, ohne bestehende Clients zu beeinflussen

```
type Beer {
  name: String!
  price: Int!
  type: BeerType!
  created: DateTime
}
```

Evolution

```
type Beer {
  name: String!
  price: Int!
  type: BeerType!
  created: DateTime

  ratings: [Rating!]
}
```

```
type Rating {
  id: ID!
  comment: String
  likes: Int
}
```

GraphQL

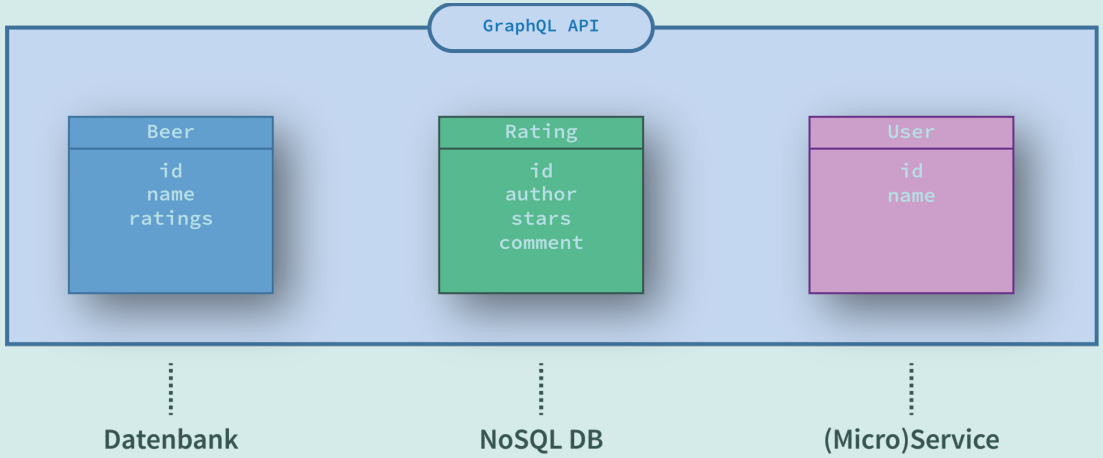
Server

Implementierung

GraphQL macht keine Aussage, wo die Daten herkommen

👉 *Ermittlung der Daten ist unsere Aufgabe*

👉 *Müssen nicht aus einer Datenbank kommen*



Implementieren von GraphQL APIs

- In der Regel muss man die Logik selbst implementieren
- Es gibt Frameworks für fast alle Programmiersprachen

IMPLEMENTIERUNG

Die Spec schreibt nicht vor, wie eine GraphQL Implementierung aussehen muss

- Es gibt aber deutliche Hinweise
- Fast alle Frameworks arbeiten danach

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)
3. Ungültige Dokumente werden abgewiesen

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)
3. Ungültige Dokumente werden abgewiesen
4. Für alle Felder werden die entsprechenden Resolver-Funktionen aufgerufen

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)
3. Ungültige Dokumente werden abgewiesen
4. Für alle Felder werden die entsprechenden Resolver-Funktionen aufgerufen
5. Die Resolver-Funktionen ermitteln die Daten für ein Feld

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)
3. Ungültige Dokumente werden abgewiesen
4. Für alle Felder werden die entsprechenden Resolver-Funktionen aufgerufen
5. Die Resolver-Funktionen ermitteln die Daten für ein Feld
6. Das Framework validiert die ermittelten Daten (Schema)

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)
3. Ungültige Dokumente werden abgewiesen
4. Für alle Felder werden die entsprechenden Resolver-Funktionen aufgerufen
5. Die Resolver-Funktionen ermitteln die Daten für ein Feld
6. Das Framework validiert die ermittelten Daten (Schema)
7. Das Framework erzeugt das Antwort-Objekt

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und Schema)
3. Ungültige Dokumente werden abgewiesen
4. Für alle Felder werden die entsprechenden Resolver-Funktionen aufgerufen
5. Die Resolver-Funktionen ermitteln die Daten für ein Feld
6. Das Framework validiert die ermittelten Daten (Schema)
7. Das Framework erzeugt das Antwort-Objekt
8. Die Antwort wird an den Client gesendet

Ablauf (stark vereinfacht)

1. HTTP Request mit Anfrage kommt an
2. Framework validiert das Dokument mit der Anfrage (Syntax und **Schema**)
3. Ungültige Dokumente werden abgewiesen
4. Für alle Felder werden die entsprechenden Resolver-Funktionen aufgerufen
5. **Die Resolver-Funktionen ermitteln die Daten für ein Feld**
6. Das Framework validiert die ermittelten Daten (Schema)
7. Das Framework erzeugt das Antwort-Objekt
8. Die Antwort wird an den Client gesendet

👉 **Die Definition des Schemas ist unsere Aufgabe**

👉 **Die Implementierung der Resolver-Funktionen ist unsere Aufgabe**

Live Beispiel : Schema + Resolver

- 🕵️ workspace-Verzeichnis
- 🕵️ schritt_01: initiales Schema
- 🕵️ schritt_02: initiale Resolver-Funktionen (BeerAdvisorGraphQlController)

QUERIES AUSFÜHREN

Die Spec schreibt nicht vor, wie Queries auszuführen sind

- Typisch: über HTTP API

- Request ist dann (meist) ein HTTP Post Request
- Response ein JSON-Objekt mit den gelesenen Daten
- Transportschicht ist eher Implementierungsdetail

QUERIES AUSFÜHREN

Die Spec schreibt nicht vor, wie Queries auszuführen sind

- Typisch: über HTTP API

- Es gibt nur einen Endpunkt für die ganze API (z.B. `/graphql`)
- Problemlos parallel zu (bestehender) REST API zu betreiben

Live Beispiel: Schema Evolution

🕵️ Query #1 ausführen

🕵️ schritt_03: Rating im Schema ergänzen

- bestehende Implementierung funktioniert weiter
- (Die Mutation ist nicht implementiert, nur als Beispiel)

🕵️ Query #1 ausführen => funktioniert immer noch unverändert

🕵️ Query #2 mit Rating ausführen => funktioniert

Live Beispiel: Fehlerbehandlung

🕵️ Beispiel technischer Fehler: addRating-Mutation ausführen (oder Query mit fehlerhaftem Feld)

"Fachliche" Fehlerbehandlung

🕵️ schritt_04: addRating-Mutation bewerten (Rückgabe-Typ)

🕵️ Exemplarisch: "@deprecated"

🕵️ schritt_04: Union-Type (02_mutation_union_type.txt)

🕵️ schritt_04: Union-Type implementierung (03_Mutation_Implementierung.txt)

Fehlerbehandlung

- Fehler(-Antworten) werden in der Regel im Schema explizit vorgesehen
- Oder im error-Feld
- HTTP Status Codes spielen eher keine Rolle

Performance von GraphQL Queries

Live Beispiel: Asynchrone Funktionen

- 🕵️ Material von 05_async "aktivieren"
- 🕵️ TracingInstrumentation einschalten
- 🕵️ Query mit beers und averageStars

- 🕵️ Problem beschreiben
- 🕵️ Umstellen auf calculateAverageStars_async im Controller

Live-Coding Beispiel: Data Loader

- 🕵️ Material von 06_data_loader: Schema und RatingController
- 🕵️ Query mit user, wird aus MicroService abgefragt
- 🕵️ Console-Log vom UserService => doppelte Ids

- 🕵️ DataLoader im RatingController aktivieren

Performance von GraphQL Queries

- Resolver Funktionen werden per Default nacheinander ausgeführt

Performance von GraphQL Queries

- Resolver Funktionen werden per Default nacheinander ausgeführt

- Frameworks unterstützen typischerweise:

    1. Asynchrone Verarbeitung

Performance von GraphQL Queries

- Resolver Funktionen werden per Default nacheinander ausgeführt

- Frameworks unterstützen typischerweise:
  1. Asynchrone Verarbeitung
  2. "Data Loader" zum Zusammenfassen von Aufrufen externer Dienste (DB, ...)

Aussagen

über

GraphQL

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

Das ist in der Absolutheit so nicht richtig

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

REST API sind konzeptionell relativ "einfach": Jeder Request liefert *eine* Ressource

- Was genau abgefragt wird, ist bereits zur *Entwicklungszeit* bekannt
- Wird vorgegeben durch das Backend

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

REST API sind konzeptionell relativ "einfach": Jeder Request liefert *eine* Ressource

- Was genau abgefragt wird, ist bereits zur *Entwicklungszeit* bekannt
- Wird vorgegeben durch das Backend

GraphQL kann mehrere Objekte liefern

- Was genau abgefragt wird, ist erst zur *Laufzeit* bekannt
- Wird vorgegeben vom Client

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

REST API sind konzeptionell relativ "einfach": Jeder Request liefert *eine* Ressource

- Was genau abgefragt wird, ist bereits zur *Entwicklungszeit* bekannt
- Wird vorgegeben durch das Backend

GraphQL kann mehrere Objekte liefern

- Was genau abgefragt wird, ist erst zur *Laufzeit* bekannt
- Wird vorgegeben vom Client

👉 **Optimierungen sind dadurch schwieriger als in REST APIs**

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

Aufwand der Entwicklung hängt an mehreren Faktoren:

- Größe und Komplexität des Schemas
- Anzahl und Art der Datenquellen
- Wie gut passen die vorhandenen Daten zum Schema

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

Größe und Komplexität des Schemas

- Extrem Beispiel: "GraphQL API, REST-Style"

```
type Beer {
  id: ID!
  name: String!
  ratings: [Rating!]
}
```

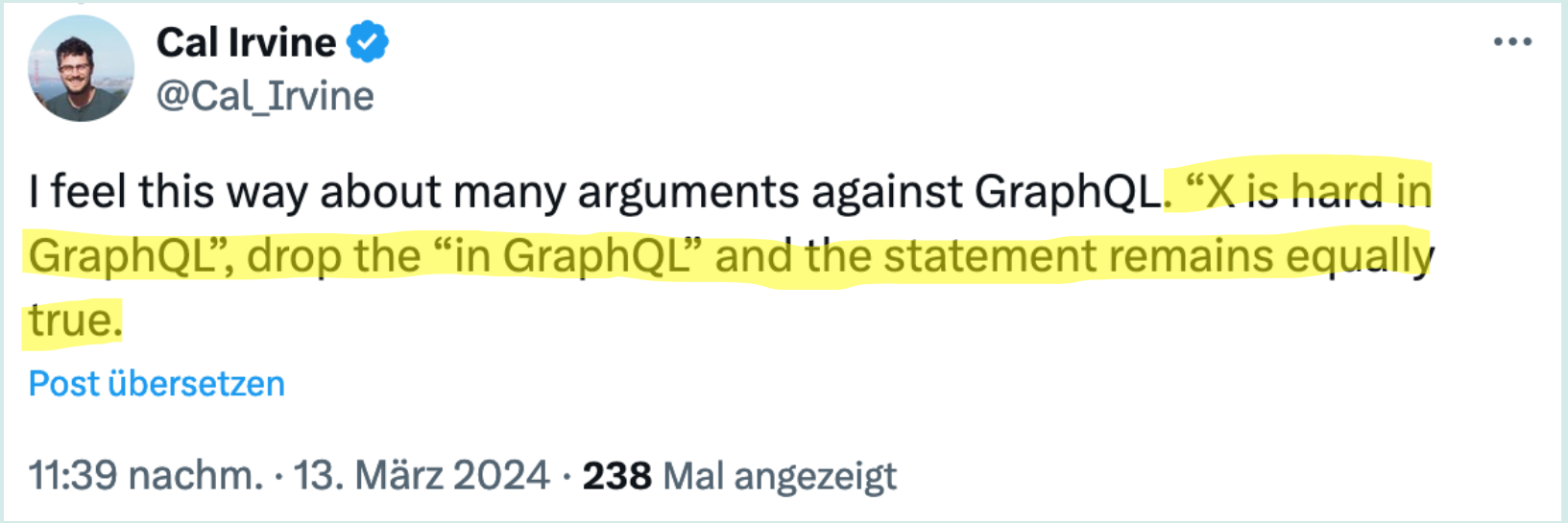
```
type Rating {
  id: ID!
  comment: String!
  stars: String!
}
```

```
type Shop {
  id: ID!
  name: String!
  address:[String!]!
}
```

```
type Query {
  beerById(id: ID!): Beer
  ratingById(id: ID!): Rating
  shopById(id: ID!): Shop
}
```

"GraphQL APIs sind komplexer in der Entwicklung als REST APIs"

Zusammenfassung



Cal Irvine ✓
@Cal_Irvine

I feel this way about many arguments against GraphQL. "X is hard in GraphQL", drop the "in GraphQL" and the statement remains equally true.

Post übersetzen

11:39 nachm. · 13. März 2024 · **238** Mal angezeigt

https://x.com/Cal_Irvine/status/1768044262124323175

"GraphQL ist SQL für's Frontend"

Das ist falsch

"GraphQL ist SQL für's Frontend"

Das ist falsch

- SQL ist viel mächtiger als die GraphQL Sprache
  - Wir haben kein SORT BY, LIMIT, JOIN etc.

"GraphQL ist SQL für's Frontend"

Das ist falsch

- SQL ist viel mächtiger als die GraphQL Sprache
  - Wir haben kein SORT BY, LIMIT, JOIN etc.
- Es werden keine fertigen Datenbank-Schema oder –Inhalte ungefiltert ausgeliefert

"GraphQL ist SQL für's Frontend"

Das ist falsch

- SQL ist viel mächtiger als die GraphQL Sprache
  - Wir haben kein SORT BY, LIMIT, JOIN etc.
- Es werden keine fertigen Datenbank-Schema oder –Inhalte ungefiltert ausgeliefert
- Daten müssen gar nicht aus einer Datenbank kommen

"GraphQL ist SQL für's Frontend"

Das ist falsch

- SQL ist viel mächtiger als die GraphQL Sprache
  - Wir haben kein SORT BY, LIMIT, JOIN etc.
- Es werden keine fertigen Datenbank-Schema oder –Inhalte ungefiltert ausgeliefert
- Daten müssen gar nicht aus einer Datenbank kommen
- Was und wie ausgeliefert wird entscheidet das Schema bzw. die Resolver-Funktionen

"GraphQL APIs sind für Faule, die kein Bock auf API Design haben"

Das ist falsch

"GraphQL APIs sind für Faule, die kein Bock auf API Design haben"

Das ist falsch

- Suggeriert wird, dass man mit GraphQL einfach alle Daten strukturlos zur Verfügung stellt

"GraphQL APIs sind für Faule, die kein Bock auf API Design haben"

Das ist falsch

- Suggeriert wird, dass man mit GraphQL einfach alle Daten strukturlos zur Verfügung stellt
- Schema-Design ist aber genauso wie in anderen API-Technologien

"GraphQL APIs sind für Faule, die kein Bock auf API Design haben"

Das ist falsch

- Suggeriert wird, dass man mit GraphQL einfach alle Daten strukturlos zur Verfügung stellt
- Schema-Design ist aber genauso wie in anderen API-Technologien
- Wie die API aussieht, bestimmen wir und nicht GraphQL

"GraphQL APIs sind für Faule, die kein Bock auf API Design haben"

Das ist falsch

- Suggeriert wird, dass man mit GraphQL einfach alle Daten strukturlos zur Verfügung stellt
- Schema-Design ist aber genauso wie in anderen API-Technologien
- Wie die API aussieht, bestimmen wir und nicht GraphQL
- Man kann sich wie bei allem anderen Mühe geben... oder eben nicht

"Mit GraphQL gibt es kein Caching"

Das ist übertrieben

- Aussage bezieht sich in der Regel auf GraphQL Anfragen per HTTP

"Mit GraphQL gibt es kein Caching"

Probleme beim Caching #1

- HTTP Post Requests nicht cachebar
  - Theoretisch ginge aber auch GET
  - Im Entwurf der neuen "GraphQL over HTTP"-Spec ist GET explizit vorgesehen

"Mit GraphQL gibt es kein Caching"

Probleme beim Caching #2

- Wenn **mehrere Objekte** abgefragt werden, muss die kürzeste Cache-Dauer gewinnen

"Mit GraphQL gibt es kein Caching"

Probleme beim Caching #2

- Wenn **mehrere Objekte** abgefragt werden, muss die kürzeste Cache-Dauer gewinnen
- Beispiel: Beer und Shop
  - Beer ist wahrscheinlich "stabil", gut cachebar
  - Shop vielleicht nicht, weil sich daran Dinge ändern können (z.B. Öffnungszeiten)

"Mit GraphQL gibt es kein Caching"

Probleme beim Caching #2

- Wenn **mehrere Objekte** abgefragt werden, muss die kürzeste Cache-Dauer gewinnen
- Beispiel: Beer und Shop
  - Beer ist wahrscheinlich "stabil", gut cachebar
  - Shop vielleicht nicht, weil sich daran Dinge ändern können (z.B. Öffnungszeiten)
- In **REST** kann jede Ressource für sich entscheiden, wie lange sie cachbar ist

"GraphQL spart Daten gegen über REST APIs"

Das ist nur teilweise korrekt

"GraphQL spart Daten gegen über REST APIs"

Das ist nur teilweise korrekt

- Richtig ist, das Netzwerk-Requests und damit Latenz gespart werden kann

"GraphQL spart Daten gegen über REST APIs"

Das ist nur teilweise korrekt

- Richtig ist, das Netzwerk-Requests und damit Latenz gespart werden kann
- Daten werden nicht automatisch de-dupliziert oder normalisiert
    - Eine Abfrage von X Ratings mit denselben Autoren liefert X mal die abgefragten Daten der Autoren
    - Das kann sparsamer sein als in REST, muss es aber nicht

"GraphQL spart Daten gegen über REST APIs"

Das ist nur teilweise korrekt

- Richtig ist, das Netzwerk-Requests und damit Latenz gespart werden kann
- Daten werden nicht automatisch de-dupliziert oder normalisiert
    - Eine Abfrage von X Ratings mit denselben Autoren liefert X mal die abgefragten Daten der Autoren
    - Das kann sparsamer sein als in REST, muss es aber nicht
- Eine "unmodified"-Antwort einer REST-Ressource liefert überhaupt keine Daten

"Das kann ich alles auch mit REST APIs machen..."

Oftmals korrekt

"Das kann ich alles auch mit REST APIs machen..."

Oftmals korrekt

- Mit Search-Parametern etc. kann man oft die Anzahl der Attribute einschränken

"Das kann ich alles auch mit REST APIs machen..."

Oftmals korrekt

- Mit Search-Parametern etc. kann man oft die Anzahl der Attribute einschränken
- Ein Schema kann man zum Beispiel mit OpenAPI beschreiben

"Das kann ich alles auch mit REST APIs machen..."

Oftmals korrekt

- Mit Search-Parametern etc. kann man oft die Anzahl der Attribute einschränken
- Ein Schema kann man zum Beispiel mit OpenAPI beschreiben
- Auch mit einer REST-API können Unterobjekte geliefert werden

"Das kann ich alles auch mit REST APIs machen..."

Oftmals korrekt

- Mit Search-Parametern etc. kann man oft die Anzahl der Attribute einschränken
- Ein Schema kann man zum Beispiel mit OpenAPI beschreiben
- Auch mit einer REST-API können Unterobjekte geliefert werden
- Mit HATEOS zeigt der Server wo bzw. wie weitere Daten geladen werden können

"Das kann ich alles auch mit REST APIs machen..."

Oftmals korrekt

- Mit Search-Parametern etc. kann man oft die Anzahl der Attribute einschränken
- Ein Schema kann man zum Beispiel mit OpenAPI beschreiben
- Auch mit einer REST-API können Unterobjekte geliefert werden
- Mit HATEOS zeigt der Server wo bzw. wie weitere Daten geladen werden können

- Aber:
  - das alles in GraphQL "eingebaut", man bekommt es aus einer Hand

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe:

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe:

- Over-/Underfetching (meines Erachtens überbewertet)

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe:

- Over-/Underfetching (meines Erachtens überbewertet)
- Schema-Evolution ermöglicht kontinuierliche Weiterentwicklung der API

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe:

- Over-/Underfetching (meines Erachtens überbewertet)
- Schema-Evolution ermöglicht kontinuierliche Weiterentwicklung der API
- Tooling und Typsicherheit bieten sehr gute Entwicklungserfahrung

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe:

- Over-/Underfetching (meines Erachtens überbewertet)
- Schema-Evolution ermöglicht kontinuierliche Weiterentwicklung der API
- Tooling und Typsicherheit bieten sehr gute Entwicklungserfahrung
- Fehlerbehandlung expliziter als mit REST

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe dagegen:

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe dagegen:

- REST ist Mainstream und in der Entwicklung gut verstanden

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe dagegen:

- REST ist Mainstream und in der Entwicklung gut verstanden
- Konsumenten wissen, wie REST funktioniert

"Wann soll ich dann GraphQL überhaupt verwenden?"

Es gibt gute Gründe dagegen:

- REST ist Mainstream und in der Entwicklung gut verstanden
- Konsumenten wissen, wie REST funktioniert
- Caching und Datensparsamkeit (kommt auf Anforderung an)

"Wann soll ich dann GraphQL überhaupt verwenden?"

it depends...

(wie immer 😇)



Vielen Dank!

Slides & Source: https://graphql.schule/softwerkskammer-nuernberg

Fragen und Kontakt: nils@nilshartmann.net

Meine Workshops: https://nilshartmann.net/workshops